

React Cheat Sheet

Core syntax & concepts at a glance

Project Setup

Create app	<code>npm create vite@latest my-app -- --template react</code>
Run dev server	<code>cd my-app && npm install && npm run dev</code>

JSX Basics

Embed expression	<code><h1>Hello, {name}!</h1></code>
Attribute / class	<code><div className="box" id={id}>...</div></code>
Fragment (no extra DOM node)	<code><><A /></></code>
Comment	<code>{/* this is a comment */}</code>

Components & Props

Function component	<code>function Welcome({ name }) { return <p>Hi {name}</p>; }</code>
Use a component	<code><Welcome name="Asha" /></code>
Children prop	<code>function Card({ children }) { return <div>{children}</div>; }</code>
Default prop value	<code>function Btn({ label = "OK" }) { ... }</code>

State & Effects

Add state	<code>const [count, setCount] = useState(0);</code>
Update state	<code>setCount(count + 1); // or setCount(c => c + 1)</code>
Run on every render	<code>useEffect(() => { ... });</code>
Run once on mount	<code>useEffect(() => { ... }, []);</code>
Run when dep changes	<code>useEffect(() => { ... }, [dep]);</code>
Cleanup on unmount	<code>useEffect(() => { const id = setInterval(fn, 1000); return () => clearInterval(id); }, []);</code>

Events

Click handler	<code><button onClick={() => setCount(count + 1)}>+1</button></code>
Form submit	<code><form onSubmit={handleSubmit}>...</form></code>
Input change	<code><input value={text} onChange={(e) => setText(e.target.value)} /></code>

Lists & Conditional Rendering

Render a list	<code>{items.map((i) => <li key={i.id}>{i.name})}</code>
Ternary	<code>{isOn ? <On /> : <Off />}</code>
Logical AND	<code>{hasError && <ErrorBanner />}</code>
Render nothing	<code>if (!visible) return null;</code>

Common Built-in Hooks

Hook	Purpose
<code>useState</code>	Add local state to a component
<code>useEffect</code>	Run side effects after render
<code>useContext</code>	Read a value from Context

useRef	Persist a value / reference a DOM node
useMemo	Memoize an expensive computed value
useCallback	Memoize a function reference
useReducer	Manage complex state via a reducer

Quick Tips

- Keep components small and focused on one job.
- Call hooks only at the top level — never in loops, conditions, or nested functions.
- Lift state up to the nearest common parent when siblings need to share it.
- Always give list items a stable, unique key.
- Never mutate state directly — always use the setter function.

React Cheat Sheet — quick reference for core syntax and patterns.